

A Preregistered Audit of Dataset Contamination Controls in LLM-for-NetOps Benchmarks

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired, confirmatory audit to test whether conservative deterministic contamination detectors (exact-match + fuzzy 5-gram + provenance heuristics) flag items in a reproducible 150-item NetOps sample and whether applying preregistered contamination controls changes validator-based success rates. Crucially, the executed sample used provenance-stable synthetic tasks (source_identifier prefix "synthetic-netops:") and shared generation semantics (independent_condition_generation = false), recorded in the archived model and task manifests; these two execution facts materially limited the audit’s sensitivity to generation-level contamination. No preregistered high-confidence contamination flags occurred and therefore no guarded exclusions or replacements were performed. All 150 baseline and matched guarded episodes passed the deterministic validator (baseline = 150/150, guarded = 150/150). The paired success-rate difference = 0.0 percentage points (95% CI [0.0, 0.0]); exact McNemar two-sided $p = 1.0$. We reconcile the preregistration→execution gap with artifact-grounded diagnosis, explain why the run is degenerate for detecting public-benchmark leakage, and provide concrete, artifact-anchored recommendations for audit designs that would be sensitive to contamination in web-sourced benchmarks.

I. INTRODUCTION

Motivation. Large language models (LLMs) are increasingly applied to NetOps tasks such as intent-to-configuration translation and automated configuration repair. Operational decisions based on benchmark results require that measured performance reflect model capability rather than dataset contamination or templated item leakage into model training. Meta-analyses and recent surveys call for contamination-aware evaluations and validator-backed oracles to avoid inflated reliability claims.

Research question and contribution. We preregistered and executed an artifact-anchored audit to answer: under conservative deterministic contamination detectors (exact normalized token equality; fuzzy 5-gram shingle overlap with preregistered thresholds; provenance timestamp heuristics) and a guarded exclusion policy, how many high-confidence flags arise in a reproducible 150-item NetOps sample, and how much does applying those controls change a single hosted LLM’s validator pass rate? Our contributions are: (1) a fully preregistered confirmatory experiment (study_id = contamination-audit-netops-2026-v1) executed end-to-end and archived; (2) exact, artifact-verified confirmatory statistics on $N_{\text{pairs}} = 150$ with deterministic validator traces; (3) an explicit preregistration→execution reconciliation that identifies why the run produced a degenerate null and prescribes artifact-grounded design changes to increase audit sensitivity.

Scope and bounded claims. The audit intentionally targeted a methodological question about preregistered contamination

controls, not an empirical prevalence estimate for public web-sourced benchmarks. The executed confirmatory sample used provenance-stable synthetic tasks (source_identifier prefix "synthetic-netops:") produced by the deterministic task generator and employed shared_initial_candidate generation semantics (independent_condition_generation = false). Because both facts are recorded in the archived model and task manifests, they limit inference: the observed zero-flag, zero-difference outcome applies to this synthetic sampling and generation regime and does not imply absence of contamination in independent public benchmarks.

II. RELATED WORK

LLM-driven NetOps systems and verifier integration. Recent system and evaluation work has investigated LLMs for intent-driven configuration synthesis and for proposing configuration repairs; these studies highlight practical value but also recurring failure modes that motivated tool-assisted or agentic architectures. Representative intent→configuration systems show that LLMs can produce syntactically valid device artifacts when guided by structured prompts or in-context examples, but that semantic correctness (reachability, policy preservation) typically requires an external validator or iterative refine-and-repair loop [<https://openalex.org/W4404602270>, 10.1109/vtc2025-fall65116.2025.11310185]. Benchmarks and controlled experiments on LLM-assisted repair report improved average repair quality when the pipeline integrates a deterministic checker (or formal verifier) with generation and repair steps; however, per-case regressions remain common and depend strongly on the verifier’s property coverage and on dataset realism [<https://openalex.org/W7157506260>, <https://openalex.org/W7163636922>].

Formal verification and deterministic oracles. The NetOps literature provides mature deterministic verification techniques (Header Space Analysis and derived tools, localized subspecification approaches, and NetKAT/related formalizations) that are practical oracles for reachability and many policy properties; these tools form the empirical basis for validator-backed evaluation in NetOps generate-validate pipelines [<http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659>, <https://openalex.org/W2742515467>, <https://openalex.org/W1882012874>]. Studies integrating such verifiers show that the choice of checks (which fields and temporal/workflow constraints are modeled) materially

changes which generation errors are visible to the auditor—an observation we incorporate by using a deterministic validator (deterministic_netops_validator_v5) that emits parse→state traces and explicit violation codes for every episode.

Agentic validate–repair architectures and their limits. Multiple contemporaneous projects evaluate agentic or multi-stage designs where an LLM proposes a candidate, a verifier checks it, and a repair generator attempts fixes when violations are found; these generate–validate–repair patterns generally improve mean repair rates versus single-pass generation but do not eliminate regressions and depend on the repair-policy and localization strategy [https://openalex.org/W7162218766, https://openalex.org/W7163636922]. Empirical studies also show scale sensitivity: effectiveness reported on curated or synthetic misconfiguration tasks often degrades when confronted with more heterogeneous, multi-vendor, or production-sourced corpora. These findings motivate two audit recommendations we make below: (1) use provenance-rich sampling to exercise contamination detectors, and (2) persist richer validator/tracer outputs to analyze subtle regression modes.

Benchmark validity, contamination, and detector strategies. Recent surveys and meta-evaluations emphasize two recurring evaluation validity risks: dataset contamination, where benchmark items (or templated prompts) are present in LLM training data and inflate measured accuracy, and construct invalidity, where tasks are overly templated or low complexity and fail to exercise operational difficulty [https://openalex.org/W4404534210, https://openalex.org/W7166900660]. The literature recommends deterministic fingerprinting (canonicalized exact matching), fuzzy n-gram overlap, and provenance/timestamp tracing as complementary heuristic detectors; these techniques form the basis for many proposed audit workflows because they are computationally tractable and interpretable. However, prior work also highlights tradeoffs: exact fingerprints miss paraphrased memorization; n-gram overlap thresholds require sensitivity analysis; and provenance heuristics depend on available source metadata and archival coverage. Our preregistration implemented a conservative combination (exact normalized equality OR fuzzy 5-gram overlap ≥ 0.80 for high confidence, with lower thresholds for sensitivity analyses) precisely because the literature documents these tradeoffs and recommends multimodal flagging and post-hoc adjudication [https://openalex.org/W4385681611, https://openalex.org/W7166900660].

How our study differs from and complements prior work. Unlike broader capability benchmarks that measure absolute LLM performance across many models, or agentic repair evaluations that emphasize mean repair rates, our study is a preregistered contamination-control audit: it asks whether a conservative, preregistered detector ensemble would produce high-confidence flags in a reproducible 150-item sample and whether deterministic guarded exclusions would alter validator pass rates. Prior benchmark and system papers typically do not publish preregistered exclusion

rules or the exact generation semantics used for paired comparisons; that omission can obscure whether observed generation discordance arises from detector sensitivity or from sampling/generation choices. By freezing detector thresholds, exclusion policies, sampling seeds, and generation semantics in a preregistration and archiving the validator traces, our experiment complements the literature by making the audit contract explicit and by demonstrating (artifact-grounded) how deterministic sampling and shared generation semantics can collapse a paired confirmatory test into a degenerate outcome. This reconciliation—linking verifier traces, provider call accounting, and the detector workflow to the resulting paired contingency—is the primary methodological contribution that complements prior empirical and system-level findings [https://openalex.org/W7157506260, https://openalex.org/W7162218766, https://openalex.org/W4404602270].

III. METHODOLOGY

Preregistration and primary estimand (design freeze). The study was preregistered under study_id = contamination-audit-netops-2026-v1 and froze the confirmatory design (estimands, sampling, detectors, exclusion/replacement rules, and analysis procedures) prior to observing model outputs. The primary estimand is the paired success-rate difference (guarded - baseline) across item×model pairs; the preregistered confirmatory test is an exact McNemar two-sided test and paired bootstrap percentile 95% CI computed with 10,000 resamples and seed = 1729. The registered analysis executor is paired_binary_analysis_v1 and the preregistration explicitly specified sensitivity runs (alternate fuzzy thresholds and failure-as-zero handling) and a multiplicity plan that treats the primary test as the single confirmatory inference.

Contamination detectors and guarded policy (operationalized). The audit implemented three deterministic detectors as preregistered defenses: (1) exact normalized token equality after canonicalization (whitespace and token normalization); (2) fuzzy 5-gram shingle overlap (Jaccard/overlap) computed on 5-gram token shingles with preregistered high-confidence threshold ≥ 0.80 and medium band [0.50, 0.80); and (3) provenance timestamp heuristics that compare item source timestamps/DOIs against the model public-cutoff metadata. The detection pipeline was implemented to produce high-confidence flags when exact equality OR 5-gram overlap ≥ 0.80 hold; medium flags were marked for overlap in [0.50,0.80) or provenance risk. The preregistered guarded policy deterministically excludes only high-confidence flagged items and replaces them with deterministic retemplated + topology-augmented items sampled from the same generator using fixed replacement seeds; replacements were required to satisfy an embedding-similarity constraint (preregistered cosine threshold ≥ 0.85) and to pass functional sanity checks before being accepted. The detector implementation persisted flagged events and replacement traces by design; the code path for detailed per-item overlap table persistence was activated

only when threshold crossings or replacements occurred (see Results for the artifact consequence).

Sampling, task generation, and generation semantics (preregistered options). The preregistration allowed sampling from a specified benchmark scope; for reproducibility, the design permitted using the deterministic task generator (deterministic_netops_task_generator_v5) with fixed generation seeds and specified source identifiers. The preregistered execution_contract allowed independent_condition_generation (two independent model calls per item, producing two independent outputs) or shared_initial_candidate semantics (single shared generation reused across conditions) depending on experimental constraints. The confirmatory sample target was task_count = 150 items (planned_episode_count = 300 episodes if independent generations were used). The sampling plan included deterministic replacement rules for excluded items and explicit seeds for replacement sampling so that paired alignment across baseline and guarded conditions is preserved in the event of exclusions.

Model and provider configuration (declared). The execution contract specified a single hosted model family for the confirmatory run (declared_model_version = "gpt-5-mini"). The model configuration used in the pipeline (documented in execution/model_configuration.json in the archive) set provider = openai_responses, max_output_tokens = 2000, maximum_attempts_per_call = 1, reasoning_effort = "minimal", and store = false. The preregistration capped maximum_model_calls = 300 (one per condition per item under independent generation), but allowed shared-generation runs to consume fewer calls when allowed by the design.

Deterministic NetOps verifier and scoring rule (oracle). The verification oracle is deterministic_netops_validator_v5 (version 5.0). The validator follows a parse $\rightarrow$ state-update $\rightarrow$ property-check pipeline: it parses candidate configuration commands into normalized command sequences, simulates the command sequence against the canonical initial state to produce an expected final state, and evaluates a set of preregistered workflow and invariant constraints (e.g., required command counts, ordered shutdown/restore patterns, protected interfaces, group availability constraints). The preregistered scoring rule is full_intent_constraint_validation: a binary pass requires reaching the intended final state while respecting all workflow constraints; the validator also records diagnostic fields per episode (normalized_configuration, parsed_command_count, required_command_count, observed_touched_field_count, violation_codes) and archives detailed traces for reproducibility. These trace fields form the deterministic basis for the paired binary outcomes used by the paired analysis executor.

Missingness, retries, and analysis handling (preregistered). The preregistered missingness plan mandated a single automated retry for model/API call failures within a five-minute window; if a retry failed, the analysis default was 'complete_pair_primary' (the pair treated per the analysis executor rules) with a sensitivity run treating failed calls as failures (zero). Exclusion rules were deterministic and applied only

to high-confidence flagged items; replacements were deterministic and seeded. The preregistration included sensitivity analyses for fuzzy thresholds at 0.50 and 0.65 and subgroup analyses by templating detector output.

Planned power and interpretation guidance (design constraints). The preregistration documented power guidance acknowledging that a two-generation per item design (independent_condition_generation = true) or a multi-model plan materially increases the number of independent trials and therefore the power to detect modest paired effects (target detect $\approx$ 5 percentage points with 80% power under conservative assumptions). In contrast, shared_generation semantics (single shared model call per item) reduce the number of independent model calls and therefore lower sensitivity to generation-level contamination unless deterministic high-confidence flags occur. These planning notes were preregistered to guide interpretation of null or degenerate paired outcomes.

Reproducibility, archival, and artifact paths. All sampling seeds, generator identifiers, validator implementations, detector code, and analysis executors were recorded and archived as part of the execution bundle. Key artifact paths produced by the pipeline include execution/task_manifest.jsonl, execution/model_configuration.json, execution/raw_results.jsonl, analysis/paired_contingency_table.csv, and analysis/contamination_summary.csv; the full execution manifest contains hashes and a complete artifact index. The preregistration SHA and the manifest entries are archived and available for verification in the execution bundle. The analysis executor parameters (bootstrap resamples = 10000, bootstrap_seed = 1729) and the exact paired_test method (exact_mcnemar_two_sided) were recorded in the analysis log and are reproduced in the archived analysis artifacts.

IV. RESULTS

Execution accounting and deterministic reconciliation. The archived execution manifest records planned_episode_count = 300 (150 items $\times$ 2 conditions) and completed_episode_count = 300 with failed_episode_count = 0. Because independent_condition_generation = false and no high-confidence exclusions occurred, the pipeline used model_calls_used = 150 (one shared generation per item) rather than two separate generations per condition; provider-call accounting and reconciliation artifacts confirm these counts and semantics (see execution/model_configuration.json, execution/task_manifest.jsonl, and analysis/deterministic_reconciliation.json). Stage-level audit fields show stage_counts: {"shared_initial_generation": 150} and condition_counts: {"shared": 150}, and provider_call_audit reports hosted_model_call_event_count = 150, completed_hosted_model_call_event_count = 150, cache_miss_count = 150, and cross_condition_cache_key_reuse_observed = false — all consistent with a single shared initial generation reused by both conditions for each item.

Contamination detectors: absence of preregistered high-confidence flags. The preregistered detectors

produced zero high-confidence contamination flags for the executed sample; the analysis artifact `analysis/contamination_summary.csv` contains no flag rows. The detector workflow was implemented to persist per-item fuzzy overlap scores only when threshold crossings or replacements were required; because no items met the preregistered high-confidence threshold (exact OR 5-gram ≥ 0.80), the overlap pipeline did not materialize an aggregated per-item overlap table in the archive. This empty flag summary is an explicit artifact result (`analysis/contamination_summary.csv`) and not an absence inferred from silence: the detector logged no threshold events to persist. Consequently, no guarded exclusions or deterministic replacements were performed (`replacements_performed = 0` in the execution manifest).

Validator outcomes and confirmatory statistics. The deterministic `netops_validator_v5` returned pass for all baseline episodes (`baseline_success_count = 150/150`; `baseline_success_rate = 1.0`) and for all guarded episodes (`guarded_success_count = 150/150`; `guarded_success_rate = 1.0`). The paired contingency table archived as `analysis/paired_contingency_table.csv` is `n11 = 150`, `n10 = 0`, `n01 = 0`, `n00 = 0`. The primary estimand (`paired_success_rate_difference_guarded_minus_baseline`) = 0.0 percentage points. The preregistered paired bootstrap percentile 95% CI (10,000 resamples, `bootstrap_seed = 1729`) is [0.0, 0.0]; the exact McNemar two-sided test yields `p = 1.0`. These numbers are computed by the archived `paired_binary_analysis_v1` executor and are recorded in `analysis/results.json` and `analysis/condition_summary.csv` (`condition,successes,failures,observed,success_rate:baseline,150,0,150,1.0; guarded,150,0,150,1.0`).

Representative episode diagnostics and difficulty coverage. Representative archived episodes span the preregistered difficulty levels (medium, hard, very_hard) and illustrate validator behavior across task families. For example, pair-000001 (medium, pattern "shutdown_configure_restore") shows `normalized_response` identical to the reference and validator fields `parsed_command_count = 4`, `required_command_count = 4`, `observed_touched_field_count = 3`, and `validation_after.valid = true`; pair-000002 (hard, "make_before_break_uplink_switchover") shows `parsed_command_count = 2` and `validation_after.valid = true`; pair-000004 (very_hard, "safe_access_service_transfer") shows `required_command_count = 4` and `validation_after.valid = true`. These representative traces are included among the archived execution/scoring artifacts and demonstrate that the validator exercised per-task workflow constraints and recorded concrete parse→state metrics for inspection. Because all episodes were scored as successes (`score = 1`, `scoring_reason_code = "VALID_CONFIGURATION"`) the global contingency counts and per-episode validator traces together indicate systematic validator agreement with the generated outputs for this synthetic sample.

Provider tracing, shared generation semantics, and implications. Provider trace fields recorded per-episode

`shared_initial_provider_trace_path` entries pointing to `execution/responses/*-shared-initial.txt` and identical `raw_response_sha256` values for baseline and guarded episodes in each pair. These traces, together with `model_calls_used = 150` and the stage_counts described above, document the operational mechanism that produced identical outputs across conditions when no exclusions were triggered. In short: when detectors produced no high-confidence flags, the guarded pipeline performed no replacements and the shared initial generation was reused for both conditions; this architecture produced perfect concordance (`n11 = 150`) and eliminated any generation-level discordance that an independent_generation design could have revealed.

Missing overlap summaries and interpretation. The preregistered fuzzy-overlap detector would have persisted per-item overlap scores and flags if thresholds were crossed. The archive contains no such per-item overlap summary because the detector implementation persisted overlap outputs only for items that triggered flags or replacements. The absence of persisted overlap scores is therefore an explicit artifact of the detector workflow combined with zero threshold crossings; it prevents reporting of min/median/percentile overlap statistics for this run, and we note this gap as an empirical limitation of the executed logging policy (see `analysis/contamination_summary.csv` and the `analysis_log.jsonl` entry describing detector events).

Synthesis of artifact evidence. The combination of (a) provenance-stable synthetic sampling (`source_identifier` values of the form "synthetic-netops:..." recorded in `execution/task_manifest.jsonl`), (b) `shared_initial_candidate` generation semantics (`execution/model_configuration.json`), (c) provider call accounting (`analysis/deterministic_reconciliation.json`), and (d) empty `contamination_summary.csv` together explain the degenerate confirmatory outcome: deterministic detectors produced no high-confidence flags and the reuse of a single shared generation per item produced identical baseline and guarded outputs, yielding complete concordance in validator outcomes. All numbers and traces cited above are archived in the execution bundle and referenced by the artifact paths given here (e.g., `execution/raw_results.jsonl`, `analysis/paired_contingency_table.csv`, `analysis/condition_summary.csv`, `analysis/contamination_summary.csv`, `analysis/deterministic_reconciliation.json`).

V. DISCUSSION

Why the result is degenerate: artifact-grounded diagnosis. Two archived execution facts explain the degenerate null outcome. First, the sampled items were provenance-stable synthetic tasks generated by `deterministic_netops_task_generator_v5` (`source_identifier` prefix "synthetic-netops:"), so canonicalized exact matches to public corpus fingerprints and high fuzzy 5-gram overlaps above the preregistered

high-confidence threshold were unlikely. Second, execution semantics set `independent_condition_generation = false` (`shared_initial_candidate`), so when no exclusions were triggered a single shared generation per item was reused for baseline and guarded episodes. Both facts are recorded in the archived `model_configuration` and `task_manifest` artifacts and together produced identical baseline and guarded outputs per item, making the paired comparison uninformative for generation-level contamination detection in this run.

What this execution answers and what it does not. The archived evidence answers the preregistered methodological question for the executed sampling regime: given provenance-stable synthetic sampling and conservative deterministic detectors, do preregistered high-confidence flags arise and do deterministic guarded exclusions alter validator pass rates? For this sample and settings the answer is no. The result does not generalize to web-sourced public benchmarks, paraphrase-sensitive contamination patterns, independent re-generation semantics, other LLM families, or multi-model designs. We therefore avoid extrapolating beyond the archive-supported conclusions.

Audit sensitivity, power, and practical tradeoffs. The pre-registration documented that a two-generation per item design (`independent_condition_generation = true`) or a multi-model plan increases the number of independent trials and therefore the power to detect paired differences of modest size (target ≈ 5 percentage points under conservative assumptions). The executed single-model, shared-generation regime reduces independent variance and makes detection of generation-level contamination implausible unless contamination flags arise deterministically. Practically, contamination audits that aim to detect web-sourced leakage should prioritize provenance-rich sampling, independent generations per condition, and multi-model replication to increase observable discordance and power.

Concrete artifact-grounded recommendations. Based on pre-registration, execution artifacts, and the deterministic reconciliation, we recommend: (1) sample provenance-rich public items (URLs/DOIs and timestamps) to exercise detector sensitivity to real-world leakage; (2) set `independent_condition_generation = true` so baseline and guarded conditions are produced independently and cannot collapse to shared outputs; (3) expand to multiple model instances and independent seeds to reveal cross-model contamination signals; (4) persist per-item fuzzy overlap scores regardless of flag status to enable overlap distribution reporting; and (5) add paraphrase-robust detectors and human adjudication for borderline cases. Each recommendation follows from artifacts: the execution manifest and deterministic reconciliation confirm the current run’s settings and limits, and the empty `contamination_summary` motivates broader sampling and independent generation in future audits.

Operational implications for NetOps. For practitioners, the audit demonstrates that audit sensitivity depends critically on sampling provenance and generation semantics. Verifier-backed validators are valuable oracles for functional

correctness checks, but auditor designs must ensure samples exercise detector capabilities and record detector outputs at sufficient granularity for post-hoc analysis. In production-risk settings, independent generation, cross-model checks, and human adjudication remain advisable.

VI. LIMITATIONS

- Single-model execution (gpt-5-mini) — results do not generalize across other LLM families, sizes, or training corpora.
- Sampled items were provenance-stable synthetic/deterministically generated tasks (source identifier prefix "synthetic-netops:") for this run; absence of high-confidence flags here may not reflect contamination prevalence in real-world, web-crawled benchmarks.
- Generation semantics used `shared_initial_candidate` across baseline and guarded conditions (`independent_condition_generation = false`), producing identical model outputs when no exclusions occurred and thus limiting sensitivity to interventions that rely on independent re-generation.
- Contamination detection relied on preregistered conservative heuristics (exact match and fuzzy 5-gram overlap thresholds); subtler contamination patterns (paraphrased memorization, partial leakage) may escape these heuristics and require richer provenance or human adjudication.
- Passing the `deterministic_netops_validator_v5` indicates satisfaction of the checked functional constraints but does not imply comprehensive production readiness or absence of semantic regressions outside the validator’s scope.

VII. CONCLUSION

We executed a fully preregistered, artifact-anchored paired audit of conservative contamination detectors for LLM-for-NetOps evaluation. In the reproducible 150-pair synthetic sample we observed zero preregistered high-confidence contamination flags and identical validator pass rates (150/150 baseline, 150/150 guarded). The degenerate null outcome is artifact-supported and stems from provenance-stable synthetic sampling and shared generation semantics; it does not imply absence of contamination in web-sourced benchmarks or failure of the preregistered detectors in different sampling regimes. We provide an explicit preregistration→execution reconciliation, confirm deterministic reconciliation checks passed, and recommend concrete design changes—provenance-rich sampling, independent generation per condition, multi-model scope, paraphrase-sensitive detectors, and matched power calculations—to increase audit sensitivity in future studies. All experiment artifacts, validator traces, analysis inputs/outputs, and execution manifests are archived and available as recorded in the Disclosure Statement below.

DISCLOSURE STATEMENT

This study was executed under the autonomous-run preregistration contamination-audit-netops-2026-v1 (preregistration

SHA-256 recorded in the archived bundle). LLM used: gpt-5-mini (declared_model_version = "gpt-5-mini"). Orchestration adapter: hosted_netops_gvr_v1. Deterministic task generator: deterministic_netops_task_generator_v5 (version 5.0). Deterministic validator: deterministic_netops_validator_v5 (version 5.0). Representative executed artifacts (by path) include execution/model_configuration.json, execution/task_manifest.jsonl, execution/raw_results.jsonl, analysis/paired_contingency_table.csv, and analysis/contamination_summary.csv; the full execution bundle contains raw responses, validator traces, execution logs, and the transformation manifest. Exact immutable initial master-prompt reference: provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Topic constraints (Generative AI and LLMs for NetOps) were selected by the human Research Director prior to the autonomy lock. After the autonomy lock, the AI system autonomously performed literature search, gap selection, preregistration, experiment execution, analysis, manuscript drafting, peer-review simulation, and revision. All generation prompts, model outputs, validator traces, sampling seeds, replacement rules, execution logs, and analysis artifacts were produced and archived by the autonomous pipeline and are included in the execution bundle. No manual human editing of technical content, analyses, references, figures, tables, captions, or the Disclosure Statement occurred after the autonomy lock. Pre-lock development and rehearsal runs occurred (permitted) but pre-lock artifacts were not used as empirical evidence for this final study unless re-created under the locked pipeline. The execution followed preregistered missingness, retry, and exclusion policies; no deviations occurred.

REFERENCES

- [1] <https://arxiv.org/abs/2604.22513>
- [2] <https://doi.org/10.48550/arxiv.2606.06212>
- [3] <https://doi.org/10.48550/arxiv.2605.22092>
- [4] <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659>
- [5] <https://doi.org/10.1007/s11704-026-60308-3>
- [6] Buğra Alperen Uluırmak, Rifat Kurban, "EvalSafetyGap: A Hybrid Survey and Conceptual Framework for LLM Evaluation-Safety Failures", 2026
- [7] Nguyen Van Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [8] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, Ting Liu, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions", 2024, 10.1145/3703155